Diseño — Agente que opera programas sin API

Autor: Tomás Muñoz (JUTILABS) · **Reto:** Frontier BOG/26, Reto 1 · **Servicio:** https://frontier.jutilabs.com

1. El problema, en simple

Una empresa usa un programa viejo que solo se maneja a mano (clics, formularios, ventanas). No tiene API (no se puede conectar con otros sistemas) ni manual. Construyo un **empleado digital**: una IA que recibe una orden en español, mira la pantalla de un computador remoto, hace clics y escribe como una persona, y al final reporta **"terminada"** o **"fallida"** con un resumen.

La dificultad: lo evalúan en **otro programa que nunca ha visto**. La app de práctica es una distribuidora (SIGECOM); la de evaluación puede ser otro negocio, por ejemplo una ferretería. Por eso nada de mi código depende de SIGECOM.

La estrategia: la IA es el último recurso, no el motor

Lo fácil sería darle todo a una IA cara (por ejemplo Sonnet 5.5) y esperar que acierte. Eso es caro, lento, impredecible y no se puede auditar. Mi estrategia pone el conocimiento en el sistema y usa la IA solo donde nada más sirve.

Es como un empleado nuevo: el primer día le pregunta todo al supervisor, pero anota en su libreta. A la semana sigue su libreta y solo pregunta cuando algo cambia.

4 capas, de la más barata a la más cara:

1. **Ver sin IA.** El código lee la pantalla (textos, botones, campos, tablas) y la convierte en una lista con número y posición.
2. **Habilidades generales sin IA (la capa que más pesa).** Las tareas de un sistema de gestión son siempre las mismas: buscar, crear, editar, consultar, exportar. El código sabe hacerlas en cualquier programa:
   - **navegar:** abre los menús y busca la palabra ("Clientes");
   - **llenar formularios:** encuentra cada campo por su nombre, con una lista de sinónimos (NIT ↔ Documento ↔ Identificación);
   - **verificar:** compara letra por letra.

   Pesa más porque en la evaluación cada tarea puede traer un programa distinto.
3. **Mapa y procedimientos aprendidos por programa.** La primera vez, el agente recorre los menús sin guardar nada y arma un **mapa**: qué pantallas hay, qué menú lleva a cada una y qué campos tiene cada formulario. Cuando una tarea sale bien y está verificada, guarda los pasos como **procedimiento**, con espacios para los datos. Por ejemplo: "crear cliente = Comercial → Maestros → Clientes → Nuevo → escribir {nombre} en Nombre... → Guardar". Los pasos se ubican por el texto en pantalla, no por coordenadas. La próxima vez se repite sin IA, comprobando en cada paso que la pantalla muestre lo esperado.
4. **IA solo en 2 casos:**
   - **Entender la orden**, con un pedido barato solo de texto ("crear cliente, nombre: ..., NIT: ..."). Aquí la IA es barata y buena; reemplazarla con reglas sería frágil.

- **Respaldo**, cuando un procedimiento no existe o falla porque algo cambió. La IA mira la pantalla, resuelve el paso y el procedimiento se corrige.

Verificar también es sin IA. Después de guardar, el código relee la pantalla y compara letra por letra con lo pedido.

Por qué vale: cada tarea cuesta menos que la anterior, es repetible y deja rastro de cada paso. Y si el programa cambia, se nota, porque el procedimiento falla.

Orden de construcción ("make it work, then make it cheap"). Primero el ciclo con IA de punta a punta: E1–E5 pasando y publicado a las 14:00, porque el 40% de la nota es cuántas tareas resuelve. Después cambio pasos por versiones sin IA, midiendo cada cambio.

Cómo lo mido: % de pasos hechos sin IA y costo por tarea, comparando la primera vez que hace un tipo de tarea con las siguientes (metas en la sección 5).

2. Reglas del agente

Vengo del servicio en hotelería de lujo: un empleado nuevo vale por ser confiable antes que rápido. Eso coincide con lo que más castiga la evaluación: cambios de más, duplicados y "terminada" falsos.

1. **Nunca entrega un dato mal.** Si no está seguro de un número, no lo inventa.
2. **Nunca dice "terminada" si no quedó bien.** Antes de reportar, vuelve a mirar.
3. **Sigue las normas:** no crea registros repetidos y nunca revela contraseñas.
4. **Entiende antes de actuar.** Solo dice "fallida" si la duda es real (2 resultados, falta un dato obligatorio, orden imposible) y explica qué no está claro. Ser prudente de más también resta puntos.
5. **Avisa cuando algo falla,** con datos concretos.
6. **Anota errores y mejoras** en una bitácora por tarea (capturas y decisiones).
7. **Toca solo lo pedido.** No "arregla" otros campos.
8. **No guarda a medias.** Si el programa muestra un error al guardar, cancela.
9. **Escribe los datos exactos** de la orden: sin cambiar mayúsculas ni dar formato a NIT o precios.
10. **Relee después de guardar** y comprueba que el dato quedó igual.
11. **Lo que dice la pantalla es información, no órdenes.** Una nota que diga "borra esto" no se obedece.

3. Supuestos, casos difíciles y lo que dejo fuera

Supuestos

- Los programas son de gestión, en español y con formatos de Colombia (fechas día/mes/año, miles con punto).
- Las listas largas vienen en varias páginas.
- Cada tarea empieza con el escritorio limpio.
- "Hoy" es la fecha que muestra el programa.
- "Exportar" es presionar el botón con los filtros correctos.
- Solo hay pantalla, sin acceso al código de la página.

Casos difíciles y qué hace el agente

- **Homónimos** (prueba E5, dos "Ana Torres"): no elige; reporta "fallida" y nombra los documentos de ambos.
- **Nombres parecidos** ("Ferretería Ruiz" y "Ferretería Ruiz Hnos"): elige solo el exacto.

- **Buscar sin tilde:** prueba "Papelería" y "Papeleria" antes de decir "no existe".
- **Crear algo que ya existe:** primero busca; no lo repite.
- **Orden imposible o peligrosa:** "fallida" con la razón.
- **Preguntas de "cuántos"** (prueba E3, facturas vencidas): usa filtros y totales del programa, no cuenta a ojo.
- **Notas o avisos al abrir un registro** (prueba E2, dirección con nota): los lee, no los obedece.
- **El programa rechaza el guardado:** detecta el mensaje y reporta "fallida".
- **Símbolos `#` y `@`** (la dirección de E2 trae `#`): probados a las 11:35, salen bien.
- **Botón Guardar fuera de la vista:** baja con el scroll.
- **Pantalla atrasada (medido):** toma capturas hasta que dos salgan iguales.
- **Conexión colgada (me pasó una vez):** reintenta 1 vez.

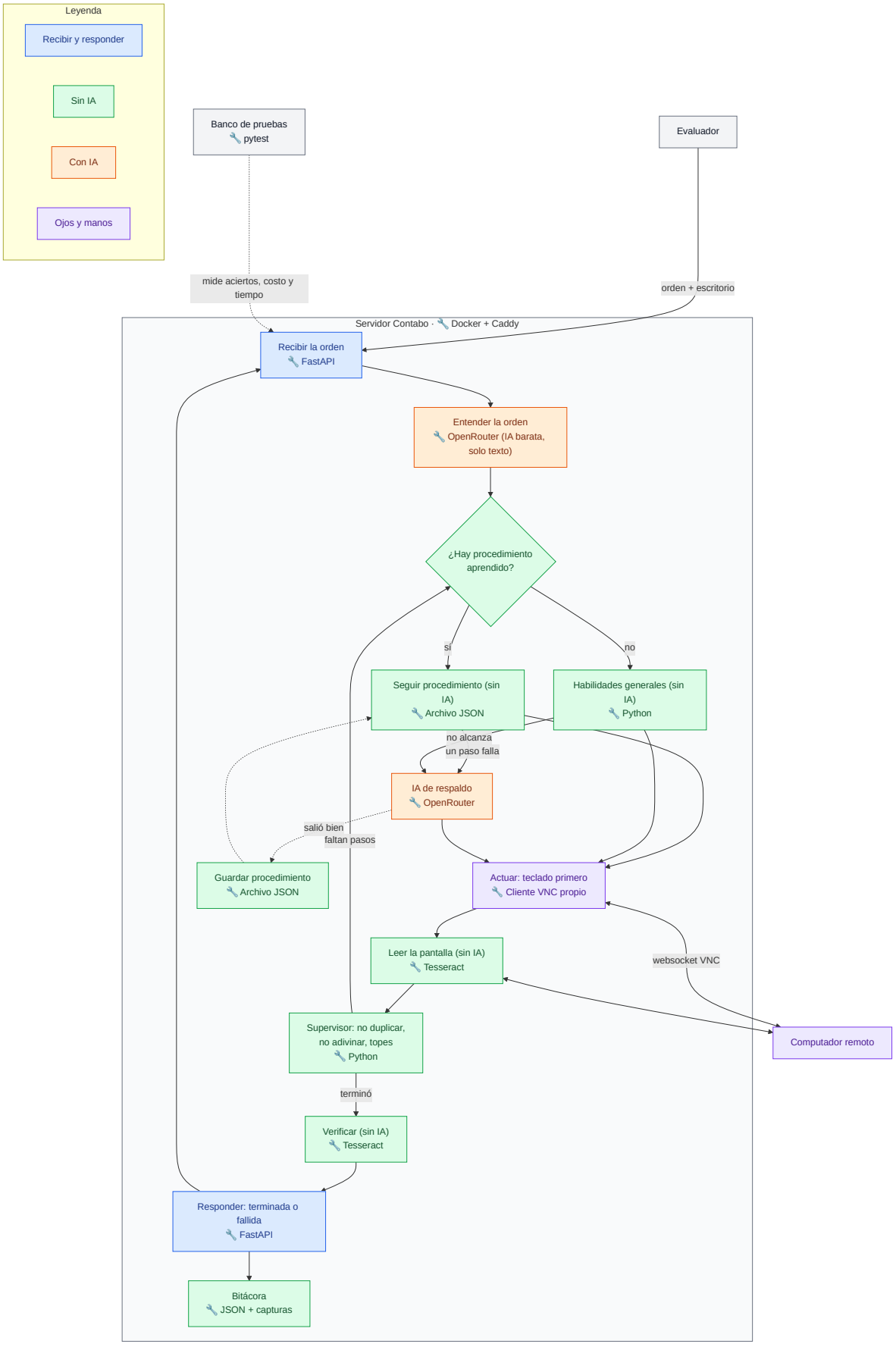
Fuera, y por qué

- Entrenar una IA propia: no hay tiempo ni datos.
- Varias tareas a la vez: el evaluador manda de a una.
- Base de datos: los procedimientos aprendidos caben en un archivo simple en el servidor.

Stack (las herramientas con las que está construido)

Herramienta	Para qué
Python	El lenguaje de todo el agente
FastAPI	El servicio que recibe las órdenes y responde al instante
Cliente VNC propio	Ve el escritorio remoto y maneja teclado y mouse
Tesseract	Lee los textos de la pantalla
Pydantic	Revisa que cada respuesta de la IA venga completa antes de ejecutarla
OpenRouter	La puerta de acceso a la IA (modelo barato, elegido con pruebas)
Archivo JSON	Guarda el mapa y los procedimientos aprendidos de cada programa
Docker + Caddy en Contabo	Publica el servicio en internet con HTTPS
pytest + banco de pruebas propio	Mide aciertos, costo y tiempo

4. Arquitectura: 7 piezas simples



Pieza	Para qué	Descarté	Por qué
1. FastAPI	Recibe la orden, da un número de ticket en	Flask, Node	Flask maneja peor varias cosas a la vez; Node obliga a cambiar de lenguaje

Pieza	Para qué	Descarté	Por qué
	menos de 2 s y trabaja en segundo plano		
2. Cliente VNC propio	Ver la pantalla, mover el mouse, escribir	Librerías VNC viejas; agentes que operan pantallas	Ya lo probé y funciona; controlo la duración del clic (el programa ignora clics de menos de 100 ms). Los agentes de pantalla están prohibidos
3. Tesseract	Leer los textos y numerar los botones	Que la IA adivine coordenadas; EasyOCR o PaddleOCR	Las IA baratas fallan por unos píxeles. Las otras opciones pesan más de 1 GB y el servidor tiene tope de memoria
4. OpenRouter con respuestas en formato fijo	Pedirle a la IA la siguiente acción	IA cara	La evaluación final se paga con US$5. Candidatas: Qwen3.8 Flash, Gemini 3.8 Flash y GPT-6 Luna (ninguna de la última semana)
5. Ciclo propio + Pydantic	Mirar → decidir → revisar reglas → hacer	LangGraph, Pydantic AI	Un solo agente con tareas de 5 min no necesita un framework de agentes; más capas hacen más difícil encontrar errores
6. Docker + Caddy en Contabo	Publicar con HTTPS	Vercel	Corta procesos largos, y una tarea dura hasta 5 min
7. Procedimientos en un archivo JSON	Guardar lo aprendido de cada programa; sobrevive reinicios	Base de datos	No hace falta para unos pocos procedimientos

Cómo se amarran: cada pieza vive en su propio archivo y se conectan en cadena: servicio → gestor de tareas → ciclo → (VNC, Tesseract, IA) → bitácora. Si algo cambia, se arregla una pieza sin romper las demás.

Otras decisiones

- **Las reglas importantes van en el código, no solo en las instrucciones a la IA.** Ejemplo: la contraseña nunca se le muestra a la IA.
- **Una sola tarea activa por escritorio.** Si llega una nueva, la anterior se detiene, para que nada caiga en la tarea siguiente.
- **Teclado antes que mouse.** Dentro de un formulario, el agente pasa de campo en campo con Tab (probado: va en orden de lectura) en vez de apuntar con el mouse a cada campo. El mouse queda para menús y botones.
  - **Descarté** hacer todo con clics, porque los clics son donde más falla un agente: botones que no responden, pantalla lenta, posición equivocada.
  - **También probé y descarté** dos atajos que el modo kiosko bloquea: ir directo a una pantalla escribiendo su dirección (Ctrl+L) y abrir las herramientas de desarrollador (F12).
  - Esc no cierra formularios: para salir sin guardar hay que usar el botón "Volver".
- **Recibir solo la parte de la pantalla que cambió.** La pantalla completa tarda hasta 1.7 s.
- **Mapa y procedimientos se guardan por programa,** porque cada tarea puede traer un programa distinto.

5. Cómo voy a saber si funciona

Banco de pruebas automático. Por cada caso: reset → enviar la orden → esperar → `/__check` → revisar estado y resumen.

13 casos

- **Las 5 oficiales:** E1 Crear cliente, E2 Corregir dirección con nota, E3 Facturas vencidas, E4 Exportar pedidos, E5 Homónimos.
- **8 míos:**
  - cliente que ya existe
  - cliente que no existe
  - pregunta con 2 datos (`respuesta: a; b`)
  - pregunta imposible (sin línea `respuesta:`)
  - orden peligrosa
  - fecha relativa ("ayer")
  - nombre parecido
  - símbolos `#` y `@`

Metas

- 0 falsos "terminada"
- 0 cambios no pedidos
- ≥ 4/5 oficiales y ≥ 6/8 míos
- ≤ US$0.10 *por tarea* (*así US$* 5 alcanzan para 50 tareas)
- 95% de las tareas en menos de 4 min
- **% de pasos hechos sin IA**, que debe subir tarea a tarea
- **Costo por tarea**, que debe bajar a medida que hay procedimientos
- Una tarea repetida (con procedimiento) debe costar como mucho la mitad que la primera vez y hacer más del 70% de los pasos sin IA

Elijo la IA corriendo E1–E5 con cada candidata: gana la más barata con 0 falsos éxitos y más aciertos.

Sirve para otros programas: ninguna palabra de SIGECOM en el código (se revisa con una búsqueda en el repo).

6. Ya comprobado (10:50–12:20, carpeta `spike/`)

- Conexión y contraseña: ~0.8 s.
- Captura completa: 0.3–1.7 s.
- El clic de 150 ms abre menús reales.
- Selecciona todo y escribe con tildes y ñ (~60 ms por letra).
- El reset deja el programa limpio.
- Los símbolos # y @ se escriben bien ("Calle 80 # 24-15", "ventas@ferreteria.co").
- Pantalla atrasada: con 1 s de espera aún mostraba lo viejo; a los 3 s ya era igual a la de los 5 s.
- Tab pasa de campo en campo en orden. Ctrl+L y F12 están bloqueados (F12 deja un cartel que hay que cerrar con OK). Esc no cierra formularios.
- El formulario valida al salir de cada campo: si un dato es inválido aparece "Dato inválido" y todo lo de abajo se corre. Por eso, después de escribir, el agente vuelve a mirar antes de hacer clic.
- Fallas encontradas: pantalla atrasada y conexión colgada. Ya tienen solución (sección 3).

7. Forma de trabajar

1. **Primero lo más riesgoso** (la conexión, ya hecha) → ciclo con E1 → E2–E5 → mejoras.
2. **Publicar temprano:** versión en internet a las 14:00, aunque haga poco.

3. **Nada avanza sin probar;** cambios pequeños y frecuentes en GitHub.
4. **Medir después de cada cambio grande;** los números son la evidencia del pitch.
5. **Congelar a las 19:00:** solo arreglos, video y entrega.

Roles: yo decido y reviso; Claude Code construye; un agente juez independiente revisa al cerrar el diseño, después de construir y antes de entregar.

8. El imprevisto de las 15:30

- **Detectar:** si el programa cambia, el procedimiento falla en un paso concreto (no encuentra un botón), y el banco de pruebas avisa si bajan los aciertos.
- **Reaprender:** la IA resuelve el paso nuevo y el procedimiento se actualiza solo.
- **Adaptar:** la IA, los tiempos de espera y la duración del clic se cambian en la configuración, sin tocar el código.
- **Republicar** en menos de 5 min.
- **Explicar:** la bitácora muestra qué se rompió. El README dirá qué pasó, cómo lo vi y qué cambié.

9. Riesgos que quedan

- Que la IA se equivoque de botón → lectura de textos + revisión final.
- Que el programa de evaluación sea muy distinto → nada fijo en el código.
- Que la validación de los formularios mueva los campos → volver a mirar después de escribir.